Who is your neighbor?

Project Overviews

"Who Is Your Neighbor?" is a casual game where the player takes on the role of a dormitory security guard. In each round, a guest approaches the entrance and presents their ID card. The player must compare the guest's appearance and identification details with the resident list to determine whether they are a real resident or a doppelganger. The core mechanic requires the player to press **ALLOW** or **DENY** based on their inspection.

Project Review

I got the idea from a game I like to play called "That's Not My Neighbor," which is also a doppelganger detection game where the player acts as a doorman, checking visitors before allowing them entry. Inspired by this, my game builds upon its core mechanics while introducing distinct new elements.

New Features

- Multiple Doppelganger Types:
  Instead of using a single disguise pattern, doppelgangers have different types of errors, such as incorrect room numbers, mismatched names, or swapped photos. This gives each round a unique challenge.
- Difficulty Selection:
  Players can choose between Easy and Hard modes before starting. In Hard mode, the differences are more subtle, making them harder to detect.

Programming Development

Game Concept

Who is Your Neighbor? is a single-screen decision-making game. The player acts as a dormitory security guard stationed at the entrance. Each round, one visitor approaches and presents an ID card. The player must compare the card against the resident list and decide whether to allow or deny entry.

Mechanics

- Visitor arrives with ID card showing: name, room number, photo, and student ID
- Player scrolls through the resident list on the right side of the screen
- Player clicks ALLOW (green) or DENY (red) button
- Correct decision: +10 points. Wrong decision: -1 life
- Game ends when all 3 lives are lost

Object-Oriented Programming Implementation

Class	Role	Key Attributes	Key Methods
Resident	Stores data of a real dormitory resident	name, room, student_id, photo_id, floor	to_dict(), validate(), display_card(), get_result()

Doppelganger	Inherits Resident; has one or more falsified fields	error_type, original_data, disguise_level	generate_error(), get_mismatch()
GameManager	Controls game loop, lives, score, round logic	lives, score, round_num, difficulty	start(), next_round(), check_decision(), game_over()
UIManager	Handles all Pygame rendering: buttons, ID card, list	screen, fonts, button_rects	draw_idcard(), draw_residentlist(), draw_hud()
StatsTracker	Records and saves gameplay statistics to CSV	session_data, filename	record_decision(), save_to_csv(), summarize()

Algorithms Involved

1. Rule-Based Verification Logic

The core algorithm compares the visitor's ID card fields against the resident database. Each field is checked independently: name, room number, ID, and photo. If any field mismatches, the visitor is flagged as a potential doppelganger. The algorithm returns a confidence score used internally to determine if the player made the correct call.

2. Doppelganger Generation Algorithm

When generating a doppelganger, the system selects a random real resident and applies one or more errors depending on the current difficulty level. Error types include: character substitution in name, room number off by ±1, photo swap with another resident, and expired ID. Higher difficulty levels use smaller, harder-to-spot errors.

3. Difficulty Scaling
The game uses an adaptive difficulty function. After every 5 rounds, the disguise_level parameter increases. At level 1, errors are obvious (completely different name). At level 3+, errors are subtle (one digit off in room number). This ensures the game remains engaging and challenging for experienced players.

4. Event-Driven Input Handling

Pygame's event loop handles all user interactions. Mouse click events are mapped to button rectangles (ALLOW/DENY) using collision detection (rect.collidepoint). Keyboard shortcuts (A = Allow, D = Deny) are also supported. Decision timestamps are captured using Python's time module for stat tracking.

5. Statistical Aggregation

After each game session, the StatsTracker class computes mean and standard deviation of decision times, calculates accuracy rate, and classifies errors by type (false allow vs false deny). Results are written to a CSV file and can be loaded for visualization using matplotlib.

Statistical Data (Prop Stats)

Data Features

The following 5 features are tracked at the per-decision level — one row is recorded every time the player clicks ALLOW or DENY. A typical 10-minute session produces 20–30 decisions. After 5 sessions, each feature will have 100+ rows of data.

Feature	Why it is good / What can it be used for	How will you obtain 100 values?	Which variable & class?	How will you display?
decision_time	Shows how fast players react. Identifies if rounds are too easy or hard based on speed.	Recorded on every ALLOW/DENY click. 20 decisions/session × 5 sessions = 100 rows.	decision_time (float) — StatsTracker	Mean, Median, Std Dev + Histogram
is_correct	Measures overall player accuracy. Useful for evaluating difficulty and tracking player improvement.	Recorded on every ALLOW/DENY click. 20 decisions/session × 5 sessions = 100 rows.	is_correct (int 0/1) — StatsTracker	Accuracy % + Line chart
error_type	Identifies which disguise type fools players most. Helps balance difficulty of each doppelganger error.	Recorded on every ALLOW/DENY click. 20 decisions/session × 5 sessions = 100 rows.	error_type (string) — Doppelganger, StatsTracker	Frequency count + Pie chart
round_number	Used as x-axis for all time-series analysis. Shows how behavior changes as the game gets harder.	Auto-increments each decision. 20 decisions/session × 5 sessions = 100 rows.	round_number (int) — GameManager, StatsTracker	X-axis in all line charts

score_at_decision	**Tracks score progression across a session. Shows how quickly players earn points.**	**Recorded on every ALLOW/DENY click. 20 decisions/session × 5 sessions = 100 rows.**	**score_at_decision (int) — GameManager, StatsTracker**	**Area chart (score per round)**

Data Recording Method

All statistical data will be saved as a CSV file using Python's built-in csv module. A new row is appended after every player's decision

Sample CSV structure:

session_id	round	decision_time	correct	error_type	score
S001	1	3.21	1	real	10
S001	2	6.54	0	room_mismatch	10
S001	3	2.88	1	name_error	20

Data is recorded in real-time using the StatsTracker class. No external database is required; CSV is sufficient for the scale of this project and enables easy import into analysis tools.

Data Analysis Report

After collecting gameplay data, the following analyses will be performed using Python (pandas + matplotlib):

Graph Planning Table

	Feature Name	Graph Objective	Graph Type	X-axis	Y-axis
Graph 1	decision_time	Show distribution of how long players take to decide per round	Histogram (Distribution)	Time (seconds)	Frequency
Graph 2	error_type	Show proportion of each doppelganger disguise type encountered	Pie Chart (Proportion)	(sections = error_type)	Percentage (%)
Graph 3	is_correct	Show how player accuracy changes across rounds over time	Line Graph (Time-series)	round_number	Accuracy (%)

Table of Statistical Values

Feature	Mean	Median	Max	Min	Std Dev
decision_time (s)	4.52	3.88	12.74	0.91	2.83
is_correct (0/1)	0.74	1	1	0	0.44
round_number	13.5	13.5	27	1	7.79
score_at_decision	74.0	75	200	0	58.6
decision_time Easy (s)	3.41	3.10	7.20	0.91	1.62

decision_time Hard (s)	**6.18**	**5.85**	**12.74**	**2.30**	**2.91**

Project Planning Timeline

Week	Week	Task
1	8	Proposal submission / Project initiation
2	9	Set up GitHub repo, project folder structure, and all 5 class skeletons
3	10	Implement Resident & Doppelganger classes with id_card system and get_result()
4	11	Build core game loop (GameManager): round logic, lives, scoring, difficulty scaling
5	12	Build UIManager: ID card display, resident list panel, ALLOW/DENY buttons, HUD
6	13	Implement StatsTracker: CSV recording, per-decision logging, post-game summary
7	14	Polish: sound, visual feedback, difficulty tuning. Collect 100+ rows of gameplay data. Prepare draft.

Document version

Version: 1.0
Date: 4 March 2025
Editor: Chayapol Kaewsakul

Date	Name	Feedback
17/03	Peraya	This is a good proposal. However, it needs some adjustment. **Minor Changes Needed** - Timeline Completion (Section 5) You cannot just submit a proposal with a 5-week gap in the schedule. A TA

		needs to see your roadmap. - Document format: Your proposal work is solid but the thing missing is easily reading and absorbing your information. Take your time adjusting the proposal. Good work after all.
30/03	Peraya	Overall, good! - You **should** clean your proposal. Format, Grammar, etc. to improve readability. - You have a Graph Plan and a Feature Table, but you are missing the Table of Statistical Values (showing Mean, Median, Max, Min, etc.). The rubric requires **at least 1 table** specifically for showing statistical values.